ENIGMA: Efficient Learning-Based Inference Guiding Machine

Jan Jakubův(✉) and Josef Urban

Czech Technical University in Prague, Prague, Czech Republic
jakubuv@gmail.com, josef.urban@gmail.com

Abstract. ENIGMA is a learning-based method for guiding given clause selection in saturation-based theorem provers. Clauses from many previous proof searches are classified as *positive* and *negative* based on their participation in the proofs. An efficient classification model is trained on this data, classifying a clause as *useful* or *un-useful* for the proof search. This learned classification is used to guide next proof searches prioritizing useful clauses among other generated clauses. The approach is evaluated on the E prover and the CASC 2016 AIM benchmark, showing a large increase of E's performance.

1 Introduction: Theorem Proving and Learning

State-of-the-art resolution/superposition automated theorem provers (ATPs) such as Vampire [16] and E [21] are today's most advanced tools for general reasoning across a variety of mathematical and scientific domains. The stronger the performance of such tools, the more realistic become tasks such as full computer understanding and automated development of complicated mathematical theories, and verification of software, hardware and engineering designs. While performance of ATPs has steadily grown over the past years due to a number of human-designed improvements, it is still on average far behind the performance of trained mathematicians. Their advanced knowledge-based proof finding is an enigma, which is unlikely to be deciphered and programmed completely manually in near future.

On large corpora such as Flyspeck, Mizar and Isabelle, the ATP progress has been mainly due to learning how to select the most relevant knowledge, based on many previous proofs [1,2,10,12]. Learning from many proofs has also recently become a very useful method for an automated finding of parameters of ATP strategies [9,17,20,23], and for a learning of sequences of tactics in interactive theorem provers (ITPs) [7]. Several experiments with the compact leanCoP [19] system have recently shown that directly using a trained machine learner for an internal clause selection can significantly prune the search space and solve additional problems [5,11,25]. An obvious next step is to implement efficient learning-based clause selection also inside the strongest superposition-based ATPs.

In this work, we introduce ENIGMA – *Efficient learNing-based Internal Guidance MAchine* for state-of-the-art saturation-based ATPs. The method applies

© Springer International Publishing AG 2017
H. Geuvers et al. (Eds.): CICM 2017, LNAI 10383, pp. 292–302, 2017.
DOI: 10.1007/978-3-319-62075-6_20

fast machine learning algorithms to a large number of proofs, and uses the trained classifier together with simpler heuristics to evaluate the millions of clauses generated during the resolution/superposition proof search. This way, the theorem prover automatically takes into account thousands of previous successes and failures that it has seen in previous problems, similarly to trained humans. Thanks to a carefully chosen efficient learning/evaluation method and its tight integration with the core ATP (in our case the E prover[1]), the penalty for this ubiquitous knowledge-based internal proof guidance is very low. This in turn very significantly improves the performance of E in terms of the number of solved problems in the CASC 2016 AIM benchmark [22].

2 Preliminaries

We use $\mathbb{N}$ to denote the set of natural numbers including 0. When S is a finite set then $|S|$ denotes its size, and S^n where $n \in \mathbb{N}$ is the n-ary Cartesian product of S, that is, the set of all vectors of size n with members from S. When $\mathbf{x} \in S^n$ then we use notation $\mathbf{x}_{[i]}$ to denote its i-th member, counting indexes from 1. We use $\mathbf{x}^T$ to denote the transposed vector.

A Multiset M over a set S is represented by a total function from S to $\mathbb{N}$, that is, $M(s)$ is the count of $s \in S$ in M. The union $M_1 \cup M_2$ of two multisets M_1 and M_2 over S is the multiset represented by the function $(M_1 \cup M_2)(s) = M_1(s) + M_2(s)$ for all $s \in S$. We use the notation $\{s_1 \mapsto n_1, \ldots, s_k \mapsto n_k\}$ to describe a multiset, omitting the members with count 0.

We assume a fixed first-order theory with stable symbol names, and denote Σ its signature, that is, a set of symbols with assigned arities. We use L to range over the set of all first-order literals (Literal), C to range over the set of all first-order clauses (Clause). Finally, we use $\mathbf{C}$ to range over sets of clauses (Clauses).

3 Training Clause Classifiers

There are many different machine learning methods, with different function spaces they can explore, different training and evaluation speeds, etc. Based on our previous experiments with premise selection and with guiding leanCoP [11,25], we have decided to choose a very fast and scalable learning method for the first ENIGMA instantiation. While more expressive learning methods usually lead to stronger single-strategy ATP results, very important aspects of our domain are that (i) the learning and proving evolve together in a feedback loop [24] where fast learning is useful, and (ii) combinations of multiple strategies – which can be provided by learning in different ways from different proofs – usually solve much more problems than the best strategy.

After several experiments, we have chosen LIBLINEAR: open source library [4] for large-scale linear classification. This section describes how we use LIBLINEAR

[1] We use E Prover 1.9.1 (http://www.eprover.org/).

to train a clause classifier to guide a given clause selection. Section 3.1 describes how training examples can be obtained from ATP runs. Section 3.2 describes how clauses are represented as fixed-length feature vectors. Finally, Sect. 3.3 describes how to use LIBLINEAR to train a clause classifier.

3.1 Extracting Training Examples from ATP Runs

Suppose we run a saturation-based ATP to prove a conjecture φ in theory T. When the ATP successfully terminates with a proof, we can extract training examples from this particular proof search as follows. We collect all the clauses that were selected as given clauses during the proof search. From these clauses, those which appear in the final proof are classified as *positives* while the remaining given clauses as *negative*. This gives us two sets of clauses, positive clauses $\mathbf{C}^{\boxplus}$ and negative clauses $\mathbf{C}^{\boxminus}$.

Re-running the proof search using the information $(\mathbf{C}^{\boxplus}, \mathbf{C}^{\boxminus})$ to prefer clauses from $\mathbf{C}^{\boxplus}$ as given clauses should significantly shorten the proof search. The challenge is to generalize this knowledge to be able to prove new problems which are in some sense similar. To achieve that, the positive and negative clauses extracted from proof runs on many related problems are combined and learned from jointly.

3.2 Encoding Clauses by Features

In order to use LIBLINEAR for linear classification (Sect. 3.3), we need to represent clauses as finite *feature vectors*. For our purposes, a feature vector $\mathbf{x}$ representing a clause C is a fixed-length vector of natural numbers whose i-th member $\mathbf{x}_{[i]}$ specifies how often the i-th feature appears in the clause C.

Several choices of clause features are possible [14], for example sub-terms, their generalizations, or paths in term trees. In this work we use term walks of length 3 as follows. First we construct a feature vector for every literal L in the clause C. We write the literal L as a tree where nodes are labeled by the symbols from Σ. In order to deal with possibly infinite number of variables and Skolem symbols, we substitute all variables and Skolem symbols with special symbols. We count for each triple of symbols $(s_1, s_2, s_3) \in \Sigma^3$, the number of directed node paths of length 3 in the literal tree, provided the trees are oriented from the root. Finally, to construct the feature vector of clause C, we sum the vectors of all literals $L \in C$.

More formally as follows. We consider a fixed theory T, hence we have a fixed signature Σ. We extend Σ with 4 special symbols for variables ($\circledast$), Skolem symbols ($\odot$), positive literals ($\oplus$), and negative literals ($\ominus$). A *feature* ϕ is a triple of symbols from Σ. The set of all features is denoted $\textsc{Feature}$, that is, $\textsc{Feature} = \Sigma^3$. *Clause (or literal) features* Φ is a multiset of features, thus recording for each feature how many times it appears in a literal or a clause. We use Φ to range over literal/clause features and the set of all literal/clause features (that is, feature multisets) is denoted $\textsc{Features}$. Recall that we represent multisets as total functions from $\textsc{Feature}$ to $\mathbb{N}$. Hence every member

$\Phi \in$ FEATURES is a total function of the type "FEATURES $\rightarrow \mathbb{N}$" and we can write $\Phi(\phi)$ to denote the count of ϕ in Φ.

Now it is easy to define function *features* of the type "LITERAL $\rightarrow$ FEATURES" which extracts features Φ from a literal L. For a literal L, we construct a rooted *feature tree* with nodes labeled by the symbols from Σ. The feature tree basically corresponds to the tree representing literal L with the following exceptions. The root node of the tree is labeled by $\oplus$ iff L is a positive literal, otherwise it is labeled by $\ominus$. Furthermore, all variable nodes are labeled by the symbol $\circledast$ and all nodes corresponding to Skolem symbols are labeled by the symbol $\odot$.

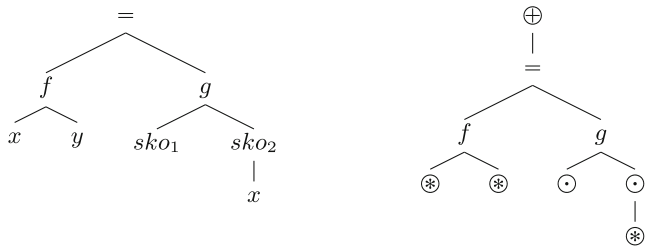

Fig. 1. The tree representing literal L_1 from Example 1 (left) and its corresponding feature tree (right).

Example 1. Consider the following equality literal $L_1 : f(x, y) = g(sko_1, sko_2(x))$ with Skolem symbols sko_1 and sko_2, and with variables x and y. In Fig. 1, the tree representation of L_1 is depicted on the left, while the corresponding feature tree used to collect features is shown on the right. □

Function *features* constructs the feature tree of a literal L and collects all directed paths of length 3. It returns the result as a feature multiset Φ.

Example 2. For literal $L_2 : P(x)$ we obtain $\mathit{features}(L_2) = \{(\oplus, P, \circledast) \mapsto 1\}$. For literal $L_3 : \neg Q(x, y)$ we have $\mathit{features}(\neg Q(x, y)) = \{(\ominus, Q, \circledast) \mapsto 2\}$. Finally, for literal L_1 from Example 1 we obtain the following multiset.

$$\{(\oplus, =, f) \mapsto 1\ ,\quad (\oplus, =, g) \mapsto 1\ ,\quad (=, f, \circledast) \mapsto 2\ ,\quad (=, g, \odot) \mapsto 2\ ,\quad (g, \odot, \circledast) \mapsto 1\}$$

□

Finally, the function *features* is extended to clauses (*features* : CLAUSE $\rightarrow$ FEATURES) by multiset union as $\mathit{features}(C) = \bigcup_{L \in C} \mathit{features}(L)$.

A Technical Note on Feature Vector Representation. In order to use LIBLINEAR, we transform the feature multiset Φ to a vector of numbers of length |FEATURE|. We assign a natural index to every feature and we construct a vector whose i-th member contains the count $\Phi(\phi)$ where i is the index of feature ϕ. Technically, we construct a bijection *sym* between Σ and $\{0, \ldots, |\Sigma| - 1\}$ which encodes symbols by natural numbers. Then we construct a bijection between

FEATURE and $\{1, \ldots, |\text{FEATURE}|\}$ which encodes features by numbers[2]. Now it is easy to construct a function *vector* which translates Φ to a vector from $\mathbb{N}^{|\text{FEATURE}|}$ as follows:

$$\mathit{vector}(\Phi) = \mathbf{x} \text{ such that } \mathbf{x}_{[\mathit{code}(\phi)]} = \Phi(\phi) \text{ for all } \phi \in \text{FEATURE}$$

3.3 Training Clause Classifiers with LIBLINEAR

Once we have the training examples $(\mathbf{C}^{\boxplus}, \mathbf{C}^{\boxminus})$ and encoding of clauses by feature vectors, we can use LIBLINEAR to construct a classification model. LIBLINEAR implements the function *train* of the type "CLAUSES × CLAUSES → MODEL" which takes two sets of clauses (positive and negative examples) and constructs a classification model. Once we have a classification model $\mathcal{M} = \mathit{train}(\mathbf{C}_{\boxplus}, \mathbf{C}_{\boxminus})$, LIBLINEAR provides a function *predict* of the type "CLAUSE×MODEL → $\{\boxplus, \boxminus\}$" which can be used to predict clause classification as positive ($\boxplus$) or negative ($\boxminus$).

LIBLINEAR supports several classification methods, but we have so far used only the default solver L2-SVM (L2-regularized L2-loss Support Vector Classification) [3]. Using the functions from the previous section, we can translate the training examples $(\mathbf{C}^{\boxplus}, \mathbf{C}^{\boxminus})$ to the set of instance-label pairs $(\mathbf{x}_i, y_i)$, where $i \in \{1, \ldots, |\mathbf{C}^{\boxplus}| + |\mathbf{C}^{\boxminus}|\}$, $\mathbf{x}_i \in \mathbb{N}^{|\text{FEATURE}|}$, $y_i \in \{\boxplus, \boxminus\}$. A training clause C_i is translated to the feature vector $\mathbf{x}_i = \mathit{vector}(\mathit{features}(C_i))$ and the corresponding y_i is set to $\boxplus$ if $C_i \in \mathbf{C}^{\boxplus}$ or to $\boxminus$ if $C_i \in \mathbf{C}^{\boxminus}$. Then, LIBLINEAR solves the following optimization problem:

$$\min_{\mathbf{w}} \big(\frac{1}{2}\mathbf{w}^T\mathbf{w} + c\sum_{i=1}^{l} \xi(\mathbf{w}, \mathbf{x_i}, y_i)\big)$$

for $\mathbf{w} \in \mathbb{R}^{|\text{FEATURE}|}$, where $c > 0$ is a penalty parameter and ξ is the following loss function.

$$\xi(\mathbf{w}, \mathbf{x_i}, y_i) = \max(1 - y_i'\mathbf{w}^T\mathbf{x_i}, 0)^2 \text{ where } y_i' = \begin{cases} 1 & \text{iff } y_1 = \boxplus \\ -1 & \text{otherwise} \end{cases}$$

LIBLINEAR implements a coordinate descend method [8] and a trust region Newton method [18].

The model computed by LIBLINEAR is basically the vector $\mathbf{w}$ obtained by solving the above optimization problem. When computing the prediction for a clause C, the clause is translated to the corresponding feature vector $\mathbf{x} = \mathit{vector}(\mathit{features}(C))$ and LIBLINEAR classifies C as positive ($\boxplus$) iff $\mathbf{w}^T\mathbf{x} > 0$. Hence we see that the prediction can be computed in time $O(\max(|\text{FEATURE}|, \mathit{length}(C)))$ where $\mathit{length}(C)$ is the length of clause C (number of symbols).

[2] We use $\mathit{code}(\phi) = \mathit{sym}(\phi_{[1]}) \cdot |\Sigma|^2 + \mathit{sym}(\phi_{[2]}) \cdot |\Sigma| + \mathit{sym}(\phi_{[3]}) + 1$.

4 Guiding the Proof Search

Once we have a LIBLINEAR model (classifier) $\mathcal{M}$, we construct a clause weight function which can be used inside the ATP given-clause loop to evaluate the generated clauses. As usual, clauses with smaller weight are selected before those with a higher weight. First, we define the function *predict* which assigns a smaller number to positively classified clauses as follows:

$$\textit{predict-weight}(C, \mathcal{M}) = \begin{cases} 1 & \text{iff } \textit{predict}(C, \mathcal{M}) = \boxplus \\ 10 & \text{otherwise} \end{cases}$$

In order to additionally prefer smaller clauses to larger ones, we add the clause length to the above predicted weight. We use $\textit{length}(C)$ to denote the length of C counted as the number of symbols. Furthermore, we use a real-valued parameter γ to multiply the length as follows.

$$\textit{weight}(C, \mathcal{M}) = \gamma \cdot \textit{length}(C) + \textit{predict-weight}(C, \mathcal{M})$$

This scheme is designed for the E automated prover which uses *clause evaluation functions* (CEFs) to select the given clause. A clause evaluation function *CEF* is a function which assigns a real weight to a clause. The unprocessed clause with the smallest weight is chosen to be the given clause. E allows combining several CEFs to jointly guide the proof search. This is done by specifying a finite number of CEFs together with their *frequencies* as follows: $(f_1 \star CEF_1, \ldots, f_k \star CEF_k)$. Each frequency f_i denotes how often the corresponding CEF_i is used to select a given clause in this weighted round-robin scheme. We have implemented learning-based guidance as a new CEF given by the above *weight* function. We can either use this new CEF alone or combine it with other CEFs already defined in E.

5 Experimental Evaluation

We use the AIM[3] category of the CASC 2016 competition for evaluation. This benchmark fits our needs as it targets internal guidance in ATPs based on training and testing examples. Before the competition, 1020 training problems were provided for the training of ATPs, while additional 200 problems were used in the competition. Prover9 proofs were provided along with all the training problems. Due to several interesting issues, we have decided not to use the training Prover9 proofs yet and instead find as many proofs as possible by a single E strategy[4].

[3] AIM is a long-term and large-scale project [15] in applied automated deduction concerned with proving open algebraic conjectures by Kinyon and Veroff.

[4] Different proof search settings (term orderings, rewriting settings, etc.) may largely change the proof search and make training examples incompatible. That is to say, a classifier trained on proofs produced with some proof search settings should be used only with the same settings. In our case, the proof search settings used to produce competition proofs are not known. Thus we resort to a single E prover strategy and generate compatible training data ourselves.

Using fast preliminary evaluation, we have selected a strong E[5] strategy S_0 (see Appendix A) which can by itself solve 239 training problems with a 30 s timeout. For comparison, E's auto-schedule mode (using optimized strategy scheduling) can solve 261 problems. We train a clause classifier model $\mathcal{M}_0$ (Sect. 3) on the 239 proofs and then run E enhanced with the classifier $\mathcal{M}_0$ in different ways to obtain even more training examples. Either we use the classifier CEF based on $\mathcal{M}_0$ (i.e., function $\mathit{weight}(C, \mathcal{M}_0)$ from Sect. 4) alone, or combine it with the CEFs from S_0 by adding $\mathit{weight}(C, \mathcal{M}_0)$ to S_0 with a grid of frequencies ranging over {1,5,6,7,8,9,10,15,20,30,40,50}. Furthermore, every combination may be run with a different value of the parameter $\gamma \in \{0, 0.1, 0.2, 0.4, 0.7, 1, 2, 4, 8\}$ of the function $\mathit{weight}(C, \mathcal{M}_0)$. All the methods are run with 30 s time limit, leading to the total of 337 solved training problems. As expected, the numbers of processed clauses and the solving times on the previously solved problems are typically very significantly decreased when using $\mathit{weight}(C, \mathcal{M}_0)$. This is a good sign, however, the ultimate test of ENIGMA's capability to learn and generalize is to evaluate the trained strategies on the testing problems. This is done as follows.

On the 337 solved training problems, we (greedily) find that 4 strategies are needed to cover the whole set. The strongest strategy is our classifier $\mathit{weight}(C, \mathcal{M}_0)$ alone with $\gamma = 0.2$, solving 318 problems. Another 15 problems are added by combining S_0 with the trained classifier $\mathit{weight}(C, \mathcal{M}_0)$ using frequency 50 and $\gamma = 0.2$. Three more new problems are solved by S_0 and two more by the trained classifier alone using $\gamma = 0$. We take these four strategies and use only the proofs they found to train a new enhanced classifier $\mathcal{M}_1$. The proofs yield 6821 positive and 219012 negative examples. Training of $\mathcal{M}_1$ by LIBLINEAR takes about 7 s – 2 s for feature extraction and 5 s for learning. The classifier evaluation on the training examples takes about 6 s and reaches 97.6% accuracy (ratio of the correctly classified clauses).

This means that both the feature generation and the model evaluation times per clause are at the order of 10 s. This is comparable to the speed at which clauses are generated by E on our hardware and evaluated by its built-in heuristics. Our learning-based guidance can thus be quickly trained and used by normal users of E, without expensive training phase or using multiple CPUs or GPUs for clause evaluation.

Then we use the $\mathcal{M}_1$ classifier to attack the 200 competition problems using 180 s time limit as in CASC. We again run several strategies: both $\mathit{weight}(C, \mathcal{M}_1)$ alone and combined with S_0 with different frequencies and parameters γ. All the strategies solve together 52 problems and only 3 of the strategies are needed for this. While S_0 solves only 22 of the competition problems, our strongest strategy solves 41 problems, see Table 1. This strategy combines S_0 with $\mathit{weight}(C, \mathcal{M}_1)$ using frequency 30 and $\gamma = 0.2$. 7 more problems are contributed by $\mathit{weight}(C, \mathcal{M}_1)$ alone with $\gamma = 0.2$ and 4 more problems are added by the E auto-schedule mode. For comparison, Vampire[6] solves 47 problems

[5] All the experiments were performed at Intel Xeon 2.3 GHz workstation.

[6] We use Vampire 4.0 in CASC mode.

Table 1. Number of problems from the 200 AIM CASC 2016 competition problems solved by differently parameterized heuristics. Different rows represent different value of γ (0,0.2,8). Columns represent different strategies (auto-schedule is E's default automated mode, 0 stands for S_0 without any Enigma guidance, ∞ stands for $\mathsf{weight}(C, \mathcal{M}_1)$ used alone, while other column labeled with f means that $\mathsf{weight}(C, \mathcal{M}_1)$ was combined with S_0 with frequency f). Entries with "-" were not run because of time restrictions.

	Auto-schedule	0	1	5	10	15	30	50	∞
0	29	22	-	-	-	-	**18**	17	16
0.2			23	31	32	40	**41**	33	40
8			23	31	31	40	**41**	33	35

(compared to our 52 proofs) with 3*180 s time limit per problem (simulating 3 runs of our best strategies, each with 180 time limit per problem).

5.1 Looping and Boosting

The recent work on the premise-selection task has shown that typically there is not a single optimal way how to guide proof search. Re-learning from new proofs as introduced by MaLARea and combining proofs and learners usually outperforms a single method. Since we are using a very fast classifier here, we can easily experiment with giving it more and different data.

First such experiment is done as follows. We add the proofs obtained on the solved 52 competition problems to the training data obtained from the 337 solved training problems. Instead of immediately re-learning and re-running (as in the MaLARea loop), we however first boost all positive examples (i.e., clauses participating in the proofs) by repeating them ten times in the training data. This way, we inform the learner to more strongly avoid misclassifying the positive examples as negative, than the other way round. The resulting clasifier $\mathcal{M}_2$ has lower overall accuracy on all of the data (93% vs. 98% for the unboosted), however, its accuracy on the relatively rare positive data grows significantly, from 12.5% to 81.8%.

Running the most successful strategy using $\mathcal{M}_2$ instead of $\mathcal{M}_1$ indeed helps. In 180 s, it solves additional 5 problems (4 of them not solved by Vampire), all of them in less than 45 s. This raises ENIGMA's performance on the competition problems to 57 problems (in general in 600 s). Interestingly, the second most useful strategy (now using $\mathcal{M}_2$ instead of $\mathcal{M}_1$) which is much more focused on doing inferences on the positively classified clauses, solves only two of these new problems, but six times faster. It is clear that we can continue experimenting this way with ENIGMA for long time, producing quickly a large number of strategies that have quite different search properties. In total we have proved 16 problems unsolved by Vampire.

6 Conclusions

The first experiments with ENIGMA are extremely encouraging. While the recent work on premise selection and on internal guidance for leanCoP indicated that large improvements are possible, this is the first practical and usable improvement of a state-of-the-art ATP by internal learning-based guidance on a large CASC benchmark. It is clear that a wide range of future improvements are possible: the learning could be dynamically used also during the proof search, training problems selected according to their similarity with the current problem,[7] more sophisticated learning and feature characterization methods could be employed, etc.

The magnitude of the improvement is unusually big for the ATP field, and similar to the improvements obtained with high-level learning in MaLARea 0.5 over E-LTB (sharing the same underlying engine) in CASC 2013 [13]. We believe that this may well mark the arrival of ENIGMAs – efficient learning-based inference guiding machines – to the automated reasoning, as crucial and indispensable technology for building the strongest automated theorem provers.

Acknowledgments. We thank Stephan Schulz for his open and modular implementation of E and its many features that allowed us to do this work. We also thank the Machine Learning Group at National Taiwan University for making LIBLINEAR openly available. This work was supported by the ERC Consolidator grant no. 649043 *AI4REASON*.

A The E Prover Strategy Used in Experiments

The following fixed E strategy S_0, described by its command line arguments, was used in the experiments:

```
--definitional-cnf=24 --destructive-er-aggressive --destructive-er
--prefer-initial-clauses -F1 --delete-bad-limit=150000000 --forward-context-sr
-winvfreqrank -c1 -Ginvfreq -WSelectComplexG --oriented-simul-paramod -tKBO6
-H(1*ConjectureRelativeSymbolWeight(SimulateSOS,0.5,100,100,100,100,1.5,1.5,1),
   4*ConjectureRelativeSymbolWeight(ConstPrio,0.1,100,100,100,100,1.5,1.5,1.5),
   1*FIFOWeight(PreferProcessed),
   1*ConjectureRelativeSymbolWeight(PreferNonGoals,0.5,100,100,100,100,1.5,1.5,1),
   4*Refinedweight(SimulateSOS,3,2,2,1.5,2))
```

References

1. Blanchette, J.C., Greenaway, D., Kaliszyk, C., Kühlwein, D., Urban, J.: A learning-based fact selector for Isabelle/HOL. J. Autom. Reasoning **57**(3), 219–244 (2016)
2. Blanchette, J.C., Kaliszyk, C., Paulson, L.C., Urban, J.: Hammering towards QED. J. Formalized Reasoning **9**(1), 101–148 (2016)

[7] In an initial experiment, a simple nearest-neighbor selection of training problems for the learning further decreases the solving times and proves one more AIM problem unsolved by Prover9.

3. Boser, B.E., Guyon, I., Vapnik, V.: A training algorithm for optimal margin classifiers. In: COLT, pp. 144–152. ACM (1992)
4. Fan, R., Chang, K., Hsieh, C., Wang, X., Lin, C.: LIBLINEAR: A library for large linear classification. J. Mach. Learn. Res. **9**, 1871–1874 (2008)
5. Färber, M., Kaliszyk, C., Urban, J.: Monte Carlo connection prover. CoRR, abs/1611.05990 (2016)
6. Gottlob, G., Sutcliffe, G., Voronkov, A. (eds.) Global Conference on Artificial Intelligence (GCAI 2015), Tbilisi, Georgia. EPiC Series in Computing, EasyChair, vol. 36, 16–19 October 2015
7. Gransden, T., Walkinshaw, N., Raman, R.: SEPIA: search for proofs using inferred automata. In: Felty, A.P., Middeldorp, A. (eds.) CADE 2015. LNCS, vol. 9195, pp. 246–255. Springer, Cham (2015). doi:10.1007/978-3-319-21401-6_16
8. Hsieh, C., Chang, K., Lin, C., Keerthi, S.S., Sundararajan, S.: A dual coordinate descent method for large-scale linear SVM. In: ICML, ACM International Conference Proceeding Series, vol. 307, pp. 408–415. ACM (2008)
9. Jakubuv, J., Urban, J.: BliStrTune: hierarchical invention of theorem proving strategies. In: Bertot, Y., Vafeiadis, V. (eds.) Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs (CPP 2017), Paris, France. pp. 43–52. ACM. 16–17 January 2017(2017)
10. Kaliszyk, C., Urban, J.: Learning-assisted automated reasoning with Flyspeck. J. Autom. Reasoning **53**(2), 173–213 (2014)
11. Kaliszyk, C., Urban, J.: FEMaLeCoP: Fairly efficient machine learning connection prover. In: Davis, M., Fehnker, A., McIver, A., Voronkov, A. (eds.) LPAR 2015. LNCS, vol. 9450, pp. 88–96. Springer, Heidelberg (2015). doi:10.1007/978-3-662-48899-7_7
12. Kaliszyk, C., Urban, J.: MizAR 40 for Mizar 40. J. Autom. Reasoning **55**(3), 245–256 (2015)
13. Kaliszyk, C., Urban, J., Vyskočil, J.: Machine learner for automated reasoning 0.4 and 0.5. CoRR, abs/1402.2359, 2014, Accepted to (PAAR 2014)
14. Kaliszyk, C., Urban, J., Vyskočil, J.: Efficient semantic features for automated reasoning over large theories. In: Yang, Q., Wooldridge, M. (eds.) IJCAI 2015, pp. 3084–3090. AAAI Press (2015)
15. Kinyon, M., Veroff, R., Vojtěchovský, P.: Loops with Abelian inner mapping groups: an application of automated deduction. In: Bonacina, M.P., Stickel, M.E. (eds.) Automated Reasoning and Mathematics. LNCS, vol. 7788, pp. 151–164. Springer, Heidelberg (2013). doi:10.1007/978-3-642-36675-8_8
16. Kovács, L., Voronkov, A.: First-order theorem proving and VAMPIRE. In: Sharygina, N., Veith, H. (eds.) CAV 2013. LNCS, vol. 8044, pp. 1–35. Springer, Heidelberg (2013). doi:10.1007/978-3-642-39799-8_1
17. Kühlwein, D., Urban, J.: MaLeS: A framework for automatic tuning of automated theorem provers. J. Autom. Reasoning **55**(2), 91–116 (2015)
18. Lin, C., Weng, R.C., Keerthi, S.S.: Trust region newton method for logistic regression. J. Mach. Learn. Res. **9**, 627–650 (2008)
19. Otten, J., Bibel, W.: leanCoP: lean connection-based theorem proving. J. Symb. Comput. **36**(1–2), 139–161 (2003)
20. Schäfer, S., Schulz, S.: Breeding theorem proving heuristics with genetic algorithms. In: Gottlob et al. [6], pp. 263–274
21. Schulz, S.: E - A Brainiac Theorem Prover. AI Commun. **15**(2–3), 111–126 (2002)
22. Sutcliffe, G.: The 8th IJCAR automated theorem proving system competition - CASC-J8. AI Commun. **29**(5), 607–619 (2016)

23. Urban, J.: BliStr: The Blind Strategymaker. In: Gottlob et al. [6], pp. 312–319
24. Urban, J., Sutcliffe, G., Pudlák, P., Vyskočil, J.: MaLARea SG1 - Machine learner for automated reasoning with semantic guidance. In: Armando, A., Baumgartner, P., Dowek, G. (eds.) IJCAR 2008. LNCS, vol. 5195, pp. 441–456. Springer, Heidelberg (2008). doi:10.1007/978-3-540-71070-7_37
25. Urban, J., Vyskočil, J., Štěpánek, P.: MaLeCoP machine learning connection prover. In: Brünnler, K., Metcalfe, G. (eds.) TABLEAUX 2011. LNCS, vol. 6793, pp. 263–277. Springer, Heidelberg (2011). doi:10.1007/978-3-642-22119-4_21